

MODULE 02 · PART 1 - FOUNDATIONS

Determinism, Ties and NULLs

Whether a query has one answer, or a set of permitted answers. You can tell before you run it.

PART 1 of 8 MODULE 2 of 26 RUNS about 30 min

Some queries have one answer. Some have **a set of permitted answers**, and the engine picks.

Two sources of ambiguity, and that is all

- ◆ **An ordering that is not total.** Two rows the ORDER BY cannot separate, so either may come first.
- ◆ **A value that is not a value.** NULL, which is neither equal nor unequal to anything, including itself.
- ◆ **Everything in this module is one of those two.** There is no third category to learn.
- ◆ **Both are silent.** Neither produces an error, ever, in any engine. That is the entire difficulty.

THE TAKEAWAY

Two causes, no errors. If you are looking for a warning to tell you a query is ambiguous, there is not one and there never will be.

On a single-threaded database, an ambiguous query was **stable by accident**.

The vendors say so, in the documentation

- ◆ **AWS states it directly for Redshift.** In any parallel system, a partial ordering leaves row order nondeterministic.
- ◆ **And that window functions may return inconsistent results** as a consequence. That is the documentation, not a caveat somebody invented.
- ◆ **Snowflake says the same thing** about window ordering. Fully deterministic only when the ORDER BY keys make each row unique.
- ◆ **This is not a bug anybody intends to fix.** It is what the relational model says, written down by the people who built the engine.

THE TAKEAWAY

When two competing vendors document the same behaviour in the same words, it is not an implementation detail. It is the contract.

An ORDER BY that is not unique is not an ordering. It is **a grouping**.

A partial order, and what it permits

```
1 SELECT sale_key, store_key, net_amount
2   FROM fact_sales_line
3  WHERE date_key = 20260831
4  ORDER BY net_amount DESC
5  LIMIT 500;
```

- 4 One column, and thousands of lines share each value. Every tie is a bundle the engine may order as it likes.
- 5 The boundary of the LIMIT almost certainly falls inside a bundle, so the last few rows are chosen by the plan.

THE TAKEAWAY

This is module one's fraud extract. Now you can say precisely what is wrong with it, rather than only that it moved.

Partial and total

PARTIAL ORDER

Rows the query cannot separate

```
1 ORDER BY net_amount DESC
2 LIMIT 500;
```

Ties are permitted to come back in any order, and the engine is behaving correctly when they do.

TOTAL ORDER

Every row distinguishable

```
1 ORDER BY net_amount DESC,
2         sale_key
3 LIMIT 500;
```

sale_key is unique, so no two rows tie on the whole clause. One permitted answer, forever.

THE TIEBREAKER

The last column in an ORDER BY must be unique, or the clause does not define an order. One column, and the query becomes reproducible.

Where a partial order does damage

- ◆ **LIMIT and TOP.** The cut falls inside a tie and the membership of the result changes.
- ◆ **ROW_NUMBER, and therefore every dedup.** Which duplicate survives is decided by the plan.
- ◆ **FIRST_VALUE and LAST_VALUE.** 'The first order' is whichever of the tied orders the engine listed first.
- ◆ **LAG and LEAD.** The previous row is only meaningful if 'previous' is defined.

THE TAKEAWAY

Every one of these is a window function or a cousin of one, which is why this module sits immediately before Part 2.

Kestrel's candidate tiebreakers

COLUMN	DOES IT MAKE AN ORDER TOTAL?
<code>sale_key</code>	Yes. Unique across the whole fact table. The default answer.
<code>txn_id</code>	No. Unique per store per day only. It looks global and is not.
<code>txn_id, line_no</code>	No. Same problem, one level deeper.
<code>sold_ts</code>	No. Second precision, and 31% of transactions tie inside one second.
<code>store_key, txn_id, line_no</code>	Yes, within a day. Across days it collides again.

THE TAKEAWAY

The second row is the trap, and it is the reason this course froze a landscape. An identifier being unique somewhere is not the same as being unique.

The uniqueness test, run once

Four lines that settle an argument permanently.

```
1 SELECT txn_id, COUNT(*) AS n
2   FROM fact_sales_line
3  GROUP BY txn_id
4  HAVING COUNT(*) > 1
5   LIMIT 10;
```

- 4 No rows returned means the column is unique. Any rows at all mean it is not, and you have examples.
- 1 Run it on the candidate, not on the documented key. The two disagree more often than anyone admits.

THE TAKEAWAY

Uniqueness is a property of the data, not of the schema. It takes one query to check and nobody checks it.

Two habits that settle it

- ◆ **Know the unique key of every table you order.** Not the primary key on the diagram. The one that is actually unique.
- ◆ **Append it to every ORDER BY that feeds a LIMIT, a ROW_NUMBER or a frame.** It costs a column.
- ◆ **If no column is unique, say so out loud.** A table with no key cannot support 'the latest row' as a concept.
- ◆ **Do not reach for DISTINCT to compensate.** It changes the row count and leaves the ordering just as partial.

THE TAKEAWAY

Ordering is the only part of SQL where the fix is reliably one column and the cost is reliably nothing. Take the deal.

Deduplicating on a tie

```
1 WITH ranked AS (  
2     SELECT p.*,  
3           ROW_NUMBER() OVER (  
4               PARTITION BY product_key  
5               ORDER BY effective_from DESC  
6           ) AS rn  
7     FROM stg_price_change p  
8 )  
9 SELECT * FROM ranked WHERE rn = 1;
```

- 5 One column. Two changes on the same day tie, so row number one is whichever the plan reached first.
- 9 The filter is correct. The ordering above it decides the answer, and decides again next run.

THE TAKEAWAY

Add the tiebreaker inside the OVER clause. An outer ORDER BY cannot repair a partition that was never ordered.

Kestrel's price feed sometimes sends two changes for one product on one day. An analyst deduplicates it with ROW_NUMBER over product_key, ordered by effective_from descending, keeping row number one.

About nine hundred products are affected on a typical day, and both changes carry the same effective_from date. The downstream margin report moves by a few thousand dollars between runs, in both directions, on data that has not changed.

What is the defect?

- ☐ A ROW_NUMBER should be RANK, so that tied rows are both kept and can be reviewed.
- ☐ B The feed should be fixed upstream. A warehouse should not receive two prices for one product per day.
- ☐ C effective_from is not unique within a product, so the partition has no total order and either row may win.
- ☐ D The partition should include store_key, because national and store-level prices are different rows.

◆ PAUSE AND DECIDE

What is the defect?

A Use RANK instead

Keeps both tied rows, so nine hundred products now produce two rows each. It replaces a wrong answer with a wrong grain.

C No total order in the partition

Both rows tie on the ordering column, so either satisfies row number one. The plan decides, and it decides again next run.

B Fix the feed upstream

Right as a principle and not available. Two price changes in one day is legitimate, and the second one is usually the correction.

D Add store_key to the partition

A real modelling point and a different one. It changes which rows compete, and the tie inside each partition survives untouched.

THE SIGNAL

"In both directions, on data that has not changed." A defect that moves a number up on Tuesday and down on Wednesday is not a data problem. It is an under-specified query.

One more thing about sorts

- ◆ **SQL sorts are not guaranteed stable.** Equal rows are not required to keep their previous relative order.
- ◆ **So you cannot sort twice to build a compound order.** The second sort is free to discard the first one's work.
- ◆ **This habit comes from spreadsheets,** where sorting is stable, and it transfers badly.
- ◆ **Put every key in one ORDER BY.** That is the only construct with a defined meaning.

THE TAKEAWAY

Two sorts are not one compound sort. If both keys matter, both keys belong in the same clause, in the order you want them applied.

NULL is not a value. It is **the absence of one**, and it behaves like nothing else in the language.

Three-valued logic

EXPRESSION	RESULT
<code>NULL = NULL</code>	UNKNOWN. Not true. Two unknowns are not known to be equal.
<code>NULL <> NULL</code>	UNKNOWN. Not true either, which surprises people more.
<code>TRUE AND UNKNOWN</code>	UNKNOWN. It might have been true.
<code>FALSE AND UNKNOWN</code>	FALSE. It cannot have been true, whatever the unknown was.
<code>TRUE OR UNKNOWN</code>	TRUE. Already satisfied, so the unknown does not matter.
<code>NOT UNKNOWN</code>	UNKNOWN. Negating ignorance leaves ignorance.

THE TAKEAWAY

Rows four and five are the useful ones. UNKNOWN can be absorbed, so the position of a predicate in an AND or OR chain changes whether NULL matters at all.

What WHERE actually does

- ◆ **WHERE keeps rows where the predicate is TRUE.** Only TRUE. That is the whole rule.
- ◆ **UNKNOWN is discarded, exactly like FALSE.** Two different things, treated identically, with no way to tell them apart afterwards.
- ◆ **Which is why 'not equal to X' does not mean 'everything else'.** It means everything else that is known.
- ◆ **HAVING, ON and CASE WHEN all follow the same rule.** One behaviour, four clauses.

THE TAKEAWAY

The mental model to replace is 'WHERE removes rows that fail'. It keeps rows that pass, and unknown does not pass.

Writing = NULL is not an error

- ◆ **It parses, it runs, and it returns nothing.** Every row evaluates to unknown, and unknown does not pass.
- ◆ **IS NULL and IS NOT NULL are the only tests** that work. They are two-valued and always return true or false.
- ◆ **Watch for it inside CASE and inside ON.** Those are where it hides, because the empty result is not the whole result.
- ◆ **COALESCE on both sides is the old workaround.** IS DISTINCT FROM is the modern one, and it does not need a sentinel value.

THE TAKEAWAY

A predicate that silently matches nothing is the worst possible failure mode, because an empty branch of a CASE just falls through to the ELSE.

The 62% that quietly disappears

```
1 SELECT COUNT(*) AS non_promo_lines
2   FROM fact_sales_line
3  WHERE promo_key <> 4471;
```

- 3 Every line with a null promo_key evaluates to unknown, not true, and is discarded.
- 1 62% of Kestrel's sales lines carry no promotion. The count returned is of the other 38%, minus one promotion.

THE TAKEAWAY

The query reads as 'lines not on promotion 4471'. It returns lines that are on some other promotion. Those are very different populations.

Comparing when one side may be absent

THE COMPARISON OPERATOR

Three-valued

```
1 WHERE promo_key <> 4471
2
3 -- or, spelled out
4 WHERE promo_key <> 4471
5     OR promo_key IS NULL
```

The first form silently drops nulls. The second is correct and every reader has to notice the second line.

THE NULL-SAFE OPERATOR

Two-valued

```
1 WHERE promo_key
2     IS DISTINCT FROM 4471
```

Returns TRUE or FALSE and never UNKNOWN. A null is simply distinct from 4471, which is what you meant.

THE TIEBREAKER

IS DISTINCT FROM is the operator people want and do not know exists. Postgres, Redshift, Snowflake, BigQuery, DuckDB, and SQL Server since 2022.

NOT IN, and the subquery with one null in it

```
1 SELECT COUNT(*)
2   FROM fact_sales_line
3  WHERE product_key NOT IN (
4      SELECT product_key
5      FROM dim_product_excluded);
```

- 5 If a single row of that subquery has a null product_key, the whole query returns no rows at all.
- 3 Not fewer rows. Not wrong rows. Zero, every time, from a query that looks entirely reasonable.

THE TAKEAWAY

The failure is total rather than partial, which is the one merciful thing about it. A result of zero gets investigated. A result that is 4% low does not.

Why NOT IN collapses

- ◆ **NOT IN expands to a chain of inequalities joined by AND.** Key not equal to the first, and not the second, and so on.
- ◆ **One of those comparisons is against NULL,** so it is UNKNOWN, and TRUE AND UNKNOWN is UNKNOWN.
- ◆ **So the chain is never TRUE.** It is UNKNOWN when the row would have qualified, and FALSE when it would not.
- ◆ **IN is safe, and NOT IN is not.** IN only needs one TRUE, and TRUE OR UNKNOWN is TRUE.

THE TAKEAWAY

The asymmetry between IN and NOT IN is row five and row three of the truth table. OR absorbs the unknown. AND does not.

NOT IN or NOT EXISTS

NOT IN

Value-based, three-valued

- Compares values with equality
- One null anywhere returns nothing
- Fails silently and totally
- Safe only on a NOT NULL column

NOT EXISTS

Row-based, two-valued

- Asks whether a matching row exists
- Nulls simply never match
- Behaves the way people expect
- Safe on any column, always

THE TIEBREAKER

NOT EXISTS is not merely safer, it is asking a different question. Does a matching row exist, rather than is this value unequal to all of those values.

Kestrel excludes staff-purchase and test products from its sales reporting using a mapping table, filtered with NOT IN against a subquery. The report has run nightly for three years.

This morning it returns zero rows. Run on its own, the subquery returns two hundred and fourteen rows, the same as yesterday plus one. The sales fact is intact and every other report against it is normal.

Why does the report return no rows?

- ☐ A The subquery column has a different data type, so no comparison can ever match.
- ☐ B NOT IN against an empty subquery returns nothing, and a filter change emptied it.
- ☐ C product_key is null on some rows of the sales fact, and those rows fail the comparison.
- ☐ D One of the two hundred and fourteen rows has a null product_key, so every test evaluates to unknown.

◆ PAUSE AND DECIDE

Why does the report return no rows?

A Data type mismatch

A mismatch that cannot be implicitly cast raises an error rather than returning an empty set. This query ran.

C Nulls on the outer side

Real, and it would drop those rows rather than all rows. Guest transactions do leave nulls in the fact, and not here.

B The subquery is empty

Backwards, and worth knowing. NOT IN against an empty set is true for every row, so the report would return everything.

D One null inside the list

The comparison chain can never be true, so no row passes. The count of rows is unchanged, which is what makes it invisible.

THE SIGNAL

"The same as yesterday plus one." The subquery is not empty and not broken. It gained one row, and that row is the whole story.

NULL is not equal to NULL. But for grouping,
it is **identical** to NULL.

Where NULL equals NULL, and where it does not

CONSTRUCT	ARE TWO NULLS THE SAME THING?
WHERE $a = b$	No. UNKNOWN, so the row is discarded.
JOIN ON $a = b$	No. Nulls never match, on either side.
GROUP BY	Yes. All nulls collapse into one group.
DISTINCT	Yes. Many null rows become one row.
PARTITION BY	Yes. All nulls form a single partition.
UNION, INTERSECT, EXCEPT	Yes. Set operations treat nulls as identical.

THE TAKEAWAY

The line runs between comparison and grouping. Anything that compares uses three-valued logic. Anything that collects treats nulls as one.

Nulls in a join key

- ◆ **A null join key matches nothing**, including another null. Those rows vanish from an inner join without comment.
- ◆ **38% of Kestrel's sales lines have no customer_key**, because guest transactions carry none. An inner join to dim_customer loses every one.
- ◆ **COALESCE on the join key is the usual fix and it is a trap.** It defeats index and hash-join optimisations on both sides.
- ◆ **Model it instead.** An 'unknown customer' dimension row with a real key removes the null and the problem together.

THE TAKEAWAY

Every dimensional model has a row for unknown, and this is why. Joining is the one operation where a null is simply lost.

NULL and the aggregates

- ◆ **COUNT(*) counts rows.** COUNT(column) counts non-null values. They are different measures.
- ◆ **SUM, AVG, MIN and MAX all skip nulls** rather than treating them as zero.
- ◆ **SUM over rows that are all null returns NULL**, not zero. That null then propagates into whatever it feeds.
- ◆ **AVG divides by the count of non-null values.** Not by the number of rows, which is where scenario three comes from.

THE TAKEAWAY

Skipping nulls is almost always the behaviour you want. The trouble is the denominator, because AVG changes its denominator without telling you.

Arithmetic propagates, and one function does not

The same missing value, two different outcomes.

```
1 SELECT net_amount - discount_amount AS margin_a,  
2        net_amount - COALESCE(discount_amount, 0)  
3                               AS margin_b  
4 FROM fact_sales_line  
5 WHERE promo_key IS NULL;
```

- 1 Null on either side makes the whole expression null. Every row in this filter returns null.
- 2 COALESCE states the assumption on the line where it is made, which is where a reader will look for it.

THE TAKEAWAY

Any arithmetic touching a nullable column returns null for those rows. It then flows into SUM, which skips it, and the total is quietly short.

Which constructs absorb a null

CONSTRUCT	BEHAVIOUR WITH A NULL INPUT
<code>a + b, a * b, a b</code>	Propagates. The whole expression becomes null.
<code>COALESCE(a, b)</code>	Absorbs. Returns the first argument that is not null.
<code>NULLIF(a, b)</code>	Creates. Returns null when the two are equal.
<code>CONCAT(a, b)</code>	Absorbs in most engines. Unlike the <code> </code> operator, which propagates.
<code>SUM, AVG, MIN, MAX</code>	Skips. Nulls are excluded before the calculation.
<code>COUNT(column)</code>	Skips. <code>COUNT(*)</code> does not, and that is the difference.

THE TAKEAWAY

Row four is the one that surprises people. The concatenation operator and the concatenation function disagree about nulls in the same engine.

CASE without ELSE returns NULL

- ◆ **A CASE expression with no ELSE and no matching branch returns null.** Not zero, not an empty string, not an error.
- ◆ **Which is how a categorisation quietly grows a null bucket** that nobody wrote and nobody counts.
- ◆ **And conditional aggregation depends on it.** SUM of a CASE with no ELSE skips the non-matching rows, which is usually what you want.
- ◆ **Write the ELSE, always.** Even ELSE null, so the next reader knows it was a decision rather than an omission.

THE TAKEAWAY

The missing ELSE is the most common way a null enters a warehouse that had none. It is created by the query rather than the data.

COUNT DISTINCT, and the value you did not count

- ◆ **COUNT(DISTINCT col) excludes nulls entirely.** Absent is not a distinct value, even though GROUP BY treats it as one.
- ◆ **So distinct promotions and promotion groups disagree by one,** consistently, and neither figure is wrong.
- ◆ **If absent is a category, count it deliberately** and say so on the slide, rather than discovering the gap in a reconciliation.
- ◆ **This is the same inconsistency as before.** COUNT DISTINCT compares. GROUP BY collects. They were always going to disagree.

THE TAKEAWAY

Distinct in a GROUP BY and distinct in a COUNT do not mean the same thing. One of them has a null group and the other does not.

Two averages over the same rows

THE AGGREGATE

Denominator is 38% of rows

```
1 SELECT AVG(discount_amount)
2   FROM fact_sales_line;
3
4 -- $3.74
```

Average discount per discounted line. A real measure, and not the one that was asked for.

WRITTEN OUT

Denominator is every row

```
1 SELECT SUM(discount_amount)
2        / COUNT(*)
3   FROM fact_sales_line;
4
5 -- $1.42
```

Average discount per line. Also a real measure, and a factor of 2.6 smaller.

THE TIEBREAKER

Neither figure is wrong. They answer different questions, and the SQL does not say which question was asked. Only the denominator does.

Kestrel's discount KPI is quoted two ways. Finance computes $\text{SUM}(\text{discount_amount})$ divided by $\text{COUNT}(*)$ and reports \$1.42. The dashboard uses $\text{AVG}(\text{discount_amount})$ and shows \$3.74. Both run over identical rows with no filters.

Non-promotional lines carry a null discount rather than a zero, and 62% of lines are non-promotional. Both teams have checked their SQL twice and can find nothing wrong with it.

Why do the two figures disagree?

- ☐ A AVG divides by the count of non-null values, so its denominator is 38% of the rows.
- ☐ B Finance is double counting, because one basket contributes many sales lines.
- ☐ C The dashboard applies a promotional filter of its own somewhere in its query layer.
- ☐ D discount_amount is negative on returns, which drags the manually computed figure down.

◆ PAUSE AND DECIDE

Why do the two figures disagree?

A AVG skips the nulls

The ratio is exactly one over the non-null fraction. 1.42 divided by 0.38 is 3.74, which is the number on the dashboard.

C A hidden filter in the dashboard

Would produce this figure by another route and is worth ruling out. The scenario states both run over identical rows.

B Finance is double counting

A genuine hazard with basket measures, and not this. Both queries use the same denominator concept, which is lines.

D Negative amounts on returns

Returns exist and move the figure by a rounding-level amount. Nothing about them produces a factor of 2.6.

THE SIGNAL

The arithmetic confirms it before you open either query. When two figures differ by exactly the reciprocal of a null rate, you have found a denominator, not a defect.

NULL has to sort somewhere. The standard
does not say where.

Where NULL sorts, by engine

ENGINE	DEFAULT TREATMENT
Redshift, Postgres, Oracle	NULL is greatest. Last in ASC, first in DESC.
Snowflake	NULL is greatest by default, and it is a session parameter.
MySQL, SQLite	NULL is smallest. First in ASC, last in DESC.
SQL Server	NULL is smallest, and there is no NULLS FIRST syntax to override it.
The SQL standard	Says nothing. Every convention above is a vendor's own decision.

THE TAKEAWAY

Kestrel's Redshift warehouse and its Postgres dashboard agree. Any tool that sorts client-side may not, and the query will look identical in both places.

It is NULLS GREATEST, not NULLS LAST

- ◆ **Postgres treats NULL as greater than everything.** Last in ASC follows from that, and first in DESC also follows.
- ◆ **So flipping to DESC moves the nulls to the top**, which is the opposite of what 'nulls last' would suggest.
- ◆ **Writing NULLS LAST explicitly is different.** Then they are last in both directions, regardless of ASC or DESC.
- ◆ **Say it as a rank, not as a position.** 'Greatest' predicts both directions. 'Last' predicts one and misleads on the other.

THE TAKEAWAY

The default is a rank. The keyword is a position. Confusing the two is how a DESC report ends up with its nulls at the top and nobody predicts it.

The same query, the nulls at either end

```
1 SELECT product_key, days_since_last_sale
2   FROM vw_product_sales_recency
3  ORDER BY days_since_last_sale DESC
4   LIMIT 50;
```

3 DESC on Redshift puts nulls first, so never-sold products occupy the top of the list.

4 1,400 products have never sold. The limit of 50 never reaches a product that has.

THE TAKEAWAY

Adding NULLS LAST is one clause. Deciding whether a never-sold product belongs in a slow-seller report is a business question, and it needs asking.

Kestrel's delist review ranks ranged products by days since last sale, descending, and takes the top fifty for merchandising to consider removing. It runs on Redshift and has been stable for a year.

After a range refresh added 1,400 new products three weeks ago, the review list came back entirely populated with products that have never recorded a sale. Merchandising nearly delisted the new range.

What is the defect?

- ☐ A days_since_last_sale should coalesce to zero for products that have never sold.
- ☐ B NULL ranks first in DESC on Redshift, so 1,400 never-sold products displaced every real candidate.
- ☐ C The report should restrict to products with at least one sale in the last twelve months.
- ☐ D Ordering ascending and taking the last fifty would place the nulls at the other end.

◆ PAUSE AND DECIDE

What is the defect?

A Coalesce to zero

Makes a never-sold product look as though it sold today. It hides genuinely dead stock, which is the opposite error and harder to notice.

C Restrict to products with sales

Removes the symptom and a category that sometimes belongs in the review. A line that sold nothing all year is exactly a delist candidate.

B NULL ranks first in DESC

Nulls are greatest on Redshift, so DESC puts them at the top. 1,400 of them is more than the fifty the report takes.

D Order ascending instead

Nulls are greatest, not last, so ascending puts them at the high end too. Taking the last fifty returns the same fifty rows.

THE SIGNAL

"Entirely populated." Not skewed, not partly wrong, but every row. A total takeover of a ranked list is the shape of a sort-position problem rather than a data problem.

Proving a query has one answer

01

Order

Is the last column of every ORDER BY unique? If not, it is a grouping.

02

Windows

Same question inside every OVER clause. Partitions need it too.

03

Nulls in

Which columns in the predicates are nullable? Check each comparison.

04

Nulls out

Any NOT IN, any AVG, any SUM that could be entirely null.

05

Assert

Run it twice and compare. Then write the uniqueness test that keeps it true.

THE TAKEAWAY

Four questions and a test. The first four take a minute on a query you already have open, and they find nearly everything in this module.

Five smells, and what each one means

WHAT YOU SEE	WHAT IT USUALLY IS
A number that moves in both directions	A partial order. Almost never a data problem.
A report that returns exactly zero rows	NOT IN against a list containing a null.
Two correct queries that disagree	Two denominators. Check AVG against SUM over COUNT.
A ranked list entirely of one category	Nulls sorting to the end you are reading.
Fewer rows than expected, by a steady share	A comparison discarding unknowns, silently.

THE TAKEAWAY

The last row is the one with no symptom. Nothing about a query that returns 38% of the rows tells you it was meant to return all of them.

Deterministic, or reproducible

DETERMINISTIC

One answer for one input

- A property of the query alone
- Decided by ordering and nulls
- Checkable before you run it
- This module is entirely about this

REPRODUCIBLE

Same answer next Tuesday

- Needs a deterministic query
- And an input that has not moved
- Late-arriving data breaks it anyway
- Module fifteen is about that half

THE TIEBREAKER

A deterministic query can still give a different answer tomorrow, because the data changed. A non-deterministic one gives a different answer on the same data, which is a different problem and a worse one.

What to write into the model, not the query

- ◆ **Declare NOT NULL where you can.** It removes an entire class of question from every query written afterwards.
- ◆ **Decide whether a measure means zero or unknown,** once, and store the one you meant.
- ◆ **A null discount and a zero discount are different claims.** One says no discount applied, the other says we do not know.
- ◆ **Fixing it per query never finishes.** Every new analyst rediscovers the same null and handles it differently.

THE TAKEAWAY

Most of this module is a modelling problem being paid for at query time, over and over, by people who did not create it.

Part 2 begins, and every window function you meet will need **a total order inside its partition.**